



معماری کامپیوتر

بهار ۱۴۰۵

استاد: دکتر سربازی آزاد

محسن صلاح - ۴۰۳۱۰۶۲۳۸

تمرین شماره ۵

۱ مقدمه

هدف از این تمرین، طراحی و پیاده‌سازی یک پردازنده ۳۲ بیتی تک‌چرخه (Single-Cycle) به زبان Verilog است که توانایی اجرای دستورات محاسباتی، انتقال داده و پرش را داشته باشد. کل پیاده‌سازی در قالب یک فایل واحد و شش ماژول مجزا انجام شده است: ماژول سطح بالا (main)، واحد کنترل (control_unit)، بانک ثبات (regfile)، واحد حساب و منطق (alu) و دو زیرماژول ترتیبی ضرب‌کننده و تقسیم‌کننده (multiplier و divider). تمام بخش‌های پردازنده به‌جز ثبات PC، بانک ثبات و همین دو زیرماژول ضرب/تقسیم کاملاً ترکیبی (Combinational) هستند؛ طبق صورت تمرین، عملگرهای ضرب و تقسیم نباید به‌صورت ترکیبی پیاده‌سازی شوند، پس این دو واحد به‌صورت ترتیبی و چندچرخه‌ای ساخته شده‌اند و در طول اجرای خود، پردازنده را موقتاً متوقف می‌کنند.

۲ واحد کنترل (control_unit)

این ماژول یک مدار ترکیبی است که با دریافت فیلدهای opcode و funct دستور جاری، تمامی سیگنال‌های کنترلی لازم برای هدایت مسیر داده را تولید می‌کند. چون هر دستور دقیقاً در یک چرخه‌ی ساعت کامل اجرا می‌شود، نیازی به جداسازی این واحد به‌صورت یک ماشین حالت سنکرون نبود؛ سیگنال‌ها فقط با یک بلوک case دو سطحی (ابتدا بر اساس opcode و در دستورات R-type نیز بر اساس funct) تعیین می‌شوند. سیگنال‌های فعال برای هر دسته از دستورات در جدول زیر آمده است.

جدول ۱: سیگنال‌های کنترلی فعال برای هر دسته از دستورات

سیگنال‌های کنترلی فعال	نمونه	دسته دستور
ALUControl، ۱=RegWrite، ۱=RegDst اساس funct بر	DIV، MUL، SUB، ADD	محاسباتی (R-type)
۱=Jr (بدون نوشتن در ثبات)	JR	پرش رجیستری (R-type)
MemWrite/MemRead، ۱=ALUSrc ۱=MemtoReg برای LW، متناظر،	SW، LW	انتقال داده (I-type)
۱=RegWrite، ۱=ALUSrc	LUI، SUBI، ADDI	محاسبه با مقدار ثابت (I-type)
SUB=ALUControl، ۱=Branch	BEQ	پرش شرطی (I-type)
۱=Jump، و علاوه بر آن ۱=Jal و ۲=MemtoReg فقط برای JAL	JAL، J	پرش (J-type)

علاوه بر سیگنال‌های ذکر شده در صورت تمرین، یک سیگنال داخلی اضافه به نام Jr هم در این واحد تعریف شده است. دلیلش این است که دستور JR، برخلاف J و JAL، به جای استفاده از فیلد آدرس ۲۶ بیتی، مستقیماً مقدار ثبات rs را به PC منتقل می‌کند؛ سیگنال Jump به‌تنهایی برای توصیف این رفتار کافی نبود.

۳ توصیف زیرمدارها

در این بخش، ساختار و منطق پیاده‌سازی سایر ماژول‌های اصلی تشکیل‌دهنده‌ی مسیر داده بررسی می‌شود.

۱.۳ ماژول بانک ثبات (regfile)

این ماژول شامل ۳۲ ثبات ۳۲ بیتی با دو درگاه خواندن ترکیبی (برای rs و rt) و یک درگاه نوشتن سنکرون است. خواندن ثبات \$0 مستقل از محتوای حافظه همواره مقدار صفر را باز می‌گرداند و نوشتن در آن نیز با شرط `WriteReg != 0` در بلوک نوشتن مسدود می‌شود. از نظر عملکرد، این روش همان کاری را انجام می‌دهد که مالتی‌پلکسر پیشنهاد شده در صورت تمرین قرار بود انجام دهد. تمامی ۳۲ ثبات همچنین به‌صورت مستقیم به خروجی‌های ماژول سطح بالا متصل شده‌اند تا وضعیت کامل ثبات‌ها در هر چرخه قابل مشاهده باشد.

۲.۳ ماژول حساب و منطق (alu) و واحدهای ترتیبی ضرب و تقسیم

ماژول alu اکنون تنها عملیات جمع، تفریق و شیفت برای LUI را به‌صورت ترکیبی انجام می‌دهد و سیگنال Zero (برای دستور BEQ) را نیز از روی همین خروجی محاسبه می‌کند. طبق صورت تمرین، عملگرهای ترکیبی ضرب و تقسیم مجاز نیستند و این دو محاسبه باید به‌صورت ترتیبی پیاده‌سازی شوند؛ به همین دلیل، دو ماژول جداگانه‌ی multiplier و divider دیگر داخل alu نیستند و مستقیماً در main نمونه‌سازی شده‌اند، چون برخلاف alu به سیگنال‌های Clk و Rst نیاز دارند.

هر دو ماژول یک ورودی Enable دارند که از برابری ALUControl با کد ضرب یا تقسیم به دست می‌آید. به محض فعال شدن Enable، عملوندها بارگذاری شده و الگوریتم به‌صورت یک بیت در هر چرخه‌ی ساعت پیش می‌رود: multiplier با روش شیفت-جمع و divider با روش تفریق بازگشتی بدون علامت (Restoring Division)، با یک رجیستر شیفتی ترکیبی ۶۴ بیتی {remainder, quotient}. هر دو محاسبه دقیقاً ۳۲ چرخه طول می‌کشد و در پایان سیگنال Done فعال می‌شود؛ نتیجه و Done تا وقتی Enable غیرفعال نشود (یعنی دستور بعدی Fetch شود) ثابت نگه داشته می‌شوند. ماژول ضرب‌کننده در ادامه آمده است:

```

1 module multiplier(
2     input Clk, Rst,
3     input Enable,
4     input [31:0] A, B,
5     output reg [31:0] Product,
6     output reg Done
7 );
8     reg [31:0] m_cand, m_pplier;
9     reg [5:0] count;
10    reg running;
11
12    always @(posedge Clk or posedge Rst) begin
13        if (Rst) begin
14            running <= 1'b0;
15            Done <= 1'b0;
16            Product <= 32'b0;

```

```

17     end else if (!Enable) begin
18         running <= 1'b0;
19         Done <= 1'b0;
20     end else if (!running && !Done) begin
21         m_cand <= A;
22         m_pplier <= B;
23         Product <= 32'b0;
24         count <= 6'd0;
25         running <= 1'b1;
26     end else if (running) begin
27         if (m_pplier[0])
28             Product <= Product + m_cand;
29         m_cand <= m_cand << 1;
30         m_pplier <= m_pplier >> 1;
31         if (count == 6'd31) begin
32             running <= 1'b0;
33             Done <= 1'b1;
34         end else begin
35             count <= count + 1;
36         end
37     end
38 end
39 endmodule

```

۴ ماژول سطح بالا و مسیر داده (main)

ماژول main، واحد کنترل، بانک ثبات و واحد حساب و منطق را به یکدیگر متصل کرده و ثبات شمارنده‌ی برنامه (PC) را نگه می‌دارد.

طبق جداول قالب دستورات در صورت تمرین (J-type و I-type، R-type)، هر فیلد دستور همیشه در یک بازه‌ی بیتی ثابت از کلمه‌ی ۳۲ بیتی قرار دارد. به همین دلیل، اولین کار در این ماژول جدا کردن این فیلدها از InstrIn با تکه‌تکه کردن ترکیبی (Bit-Slicing) بردار است. تمام این فیلدها بدون توجه به نوع واقعی دستور جاری هم‌زمان استخراج می‌شوند و این واحد کنترل است که بر اساس opcode و funct تشخیص می‌دهد کدام یک از آن‌ها برای دستور فعلی معنا دارد. مقدار ۱۶ بیتی imm هم با تکرار بیت علامتش (بیت ۱۵) به یک مقدار ۳۲ بیتی بسط‌علامت‌دار (sext_imm) تبدیل می‌شود تا در محاسبات بعدی، مثل آدرس‌دهی SW/LW یا افست BEQ، قابل استفاده باشد:

```

1 wire [5:0] opcode = InstrIn[31:26];
2 wire [4:0] rs = InstrIn[25:21];
3 wire [4:0] rt = InstrIn[20:16];
4 wire [4:0] rd = InstrIn[15:11];
5 wire [5:0] funct = InstrIn[5:0];
6 wire [15:0] imm = InstrIn[15:0];
7 wire [25:0] jaddr = InstrIn[25:0];
8
9 wire [31:0] sext_imm = {{16{imm[15]}}}, imm};

```

بخش بعدی این ماژول، تعیین نشانی دستوری است که در چرخه‌ی بعد باید واکنشی شود. در حالت عادی این

مقدار برابر 4 + PC است، اما برخی دستورهای صورت تمرین این مقدار پیش فرض را تغییر می دهند: دستور JR مستقیماً مقدار ثابت rs را جایگزین PC می کند، دستورهای J و JAL نشانی را از فیلد ۲۶ بیتی آدرس می سازند، و دستور BEQ تنها در صورت برابر بودن دو ثابت (یعنی فعال بودن هم زمان سیگنال های Branch و Zero) به نشانی محاسبه شده از افست پرش منتقل می شود. این بخش از next_PC با یک مالتی پلکسر چهارحالتی از میان این گزینه ها انتخاب می شود؛ اولویت انتخاب نیز دقیقاً به همین ترتیب است: ابتدا JR، سپس Jump (برای J/JAL)، با نشانی jump_target)، سپس پرش شرطی فعال (با نشانی branch_target)، و در نهایت در صورت نبود هیچ کدام، مقدار پیش فرض 4 + PC. چون در هر چرخه حداکثر یکی از این شرایط می تواند برقرار باشد، این اولویت بندی هیچگاه باعث ابهام در انتخاب نمی شود؛ jump_target و branch_target نیز دقیقاً مطابق روابط ارائه شده در صورت تمرین محاسبه می شوند.

از آنجا که طبق صورت تمرین ضرب و تقسیم باید به صورت ترتیبی پیاده سازی شوند، این دو عملیات دیگر در یک چرخه به پایان نمی رسند. برای هماهنگی با این موضوع، یک سیگنال stall در main تعریف شده که تا وقتی دستور جاری MUL یا DIV باشد و سیگنال Done متناظرش هنوز فعال نشده باشد، فعال می ماند. تا زمانی که stall فعال است، به جای مقدار محاسبه شده ی بالا، خود PC به عنوان next_PC انتخاب می شود (یعنی PC ثابت می ماند) و سیگنال واقعی نوشتن در بانک ثبات نیز با شرط stall && !RegWrite غیر فعال می شود تا مقدار ناقص میانی محاسبه زودتر از موعد در ثبات مقصد نوشته نشود. به محض فعال شدن Done، stall صفر شده و در همان چرخه هم PC به دستور بعدی می رود و هم نتیجه ی نهایی در ثبات مقصد نوشته می شود. ثبات PC در لبه ی بالارونده ی ساعت با همین next_PC به روزرسانی می شود.

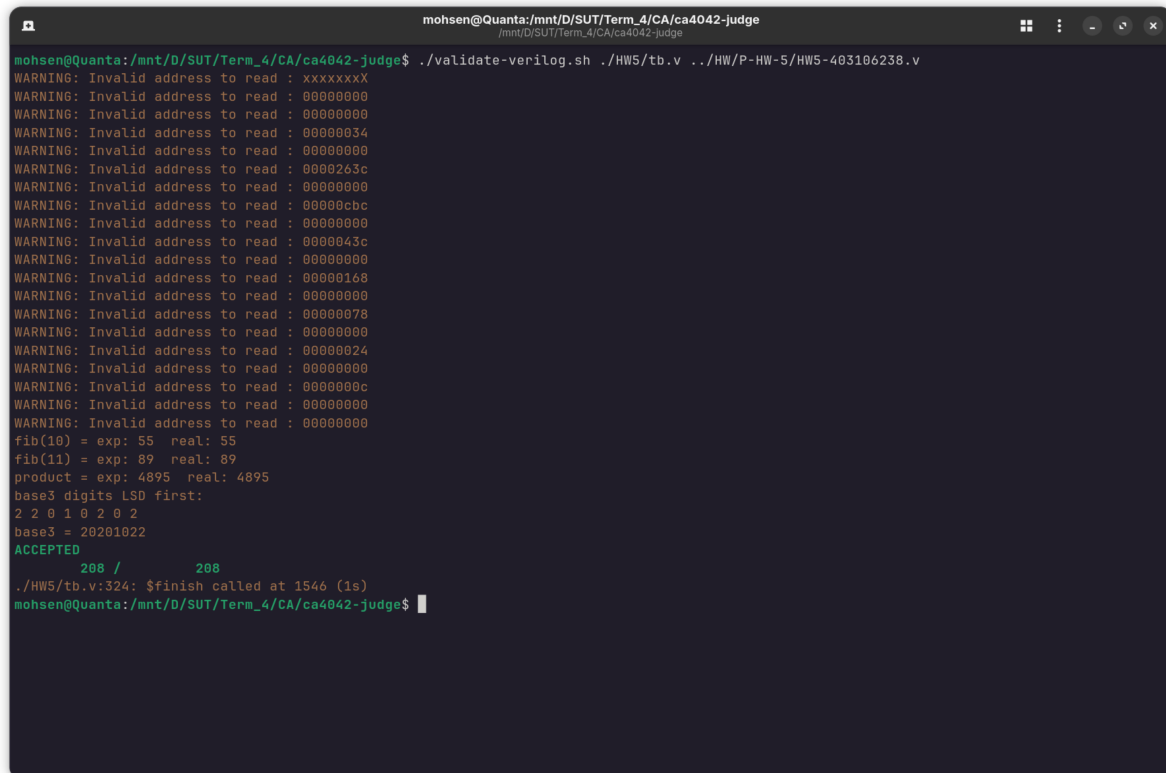
در صورت تمرین ذکر شده که دو بیت کم ارزش (همواره صفر) نشانی هنگام آدرس دهی به حافظه باید نادیده گرفته شوند. از این رو خروجی های InstrAddr و DataAddr با شیفت دو بیت به راست نسبت به مقدار داخلی PC و خروجی نهایی بخش محاسباتی تولید می شوند:

$$\text{InstrAddr} = \text{PC} \gg 2 \quad \text{DataAddr} = \text{final_result} \gg 2$$

که در آن final_result بر اساس ALUControl، از میان خروجی alu، multiplier و divider انتخاب می شود. البته تمام محاسبات داخلی نشانی، مثل آدرس پرش های J/JAL و BEQ، بر اساس نشانی بیتی (مضرب ۴) انجام می شود و این تبدیل فقط در مرز خروجی به حافظه اعمال می شود.

۵ اجرای آزمون

برای اطمینان از صحت پردازنده‌ی طراحی‌شده، برنامه‌ی آزمون استاندارد صورت تمرین (شامل محاسبه‌ی دنباله‌ی فیبوناتچی، ضرب دو مقدار و تبدیل حاصل ضرب به مبنای سه) با تست‌بنچ رسمی تمرین روی مدار اجرا شد. مقدار هر ثبات در پایان اجرا با مقدار محاسبه‌شده توسط مدار معیار مقایسه شد. خروجی موفقیت‌آمیز اجرای تست‌بنچ در شکل زیر مشاهده می‌شود.



```
mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge$ ./validate-verilog.sh ./HWS/tb.v ../HW/P-HW-5/HW5-403106238.v
WARNING: Invalid address to read : xxxxxxxX
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000034
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 0000263c
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 0000cbbc
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 0000043c
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000168
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000078
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000024
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 0000000c
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
fib(10) = exp: 55 real: 55
fib(11) = exp: 89 real: 89
product = exp: 4895 real: 4895
base3 digits LSD first:
2 2 0 1 0 2 0 2
base3 = 20201022
ACCEPTED
208 / 208
./HWS/tb.v:324: $finish called at 1546 (1s)
mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge$
```

شکل ۱: اجرای تست‌بنچ تمرین